

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO DỰ ÁN CUỐI KỲ

Ứng dụng LLM giải Toán Olympic
AI Mathematical Olympiad
(Kaggle)

Sinh viên thực hiện:

- Nguyễn Thị Nhật Minh
- Phạm Thành Trung

Môn học: [Học sâu]

Hà Nội, tháng 06/2026

Mục lục

1	PHÁT BIỂU BÀI TOÁN VÀ PHẠM VI DỰ ÁN	1
1.1	Phát biểu bài toán	1
1.2	Phạm vi dự án	1
1.3	Phân chia công việc	2
2	CƠ SỞ LÝ THUYẾT VÀ PHƯƠNG PHÁP	2
2.1	Mô hình ngôn ngữ lớn (Large Language Model)	2
2.2	Program-of-Thought (PoT)	3
2.3	Self-Consistency và Majority Voting	3
2.4	Kiến trúc mô hình Qwen2.5-Math	3
3	THIẾT KẾ HỆ THỐNG	4
3.1	Kiến trúc tổng thể	4
3.2	Module Prompt Builder	4
3.3	Module Engine	5
3.4	Module Parser	5
3.5	Module Python Executor	5
3.6	Module Majority Voting	6
3.7	Cơ chế dự phòng và Xử lý ngoại lệ tổng thể (Fallback Mechanism) . . .	6
4	THỰC NGHIỆM VÀ KẾT QUẢ	7
4.1	Môi trường thực nghiệm	7
4.2	Bộ dữ liệu	7
4.3	Thiết lập thực nghiệm	8
4.4	Kết quả thực nghiệm	8
4.5	Đánh giá ảnh hưởng của Majority Voting	9
4.6	Thảo luận	10
5	PHÂN TÍCH LỖI	10
5.1	Phân loại lỗi	10
5.2	Formatting Error	10
5.3	Logical Error	11
5.4	Algorithmic Error	11
5.5	System Error	12
5.6	Đánh giá tổng quát	12
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	12
6.1	Kết luận	12
6.2	Hướng phát triển	13

Danh sách hình vẽ

Danh sách bảng

1	Phân công công việc của các thành viên	2
2	Môi trường thực nghiệm	7
3	Các tham số thực nghiệm	8
4	Kết quả thực nghiệm (mẫu 10 bài toán)	9
5	Phân loại các nhóm lỗi	10

1 PHÁT BIỂU BÀI TOÁN VÀ PHẠM VI DỰ ÁN

1.1 Phát biểu bài toán

Trong những năm gần đây, các mô hình ngôn ngữ lớn (Large Language Models – LLMs) đã đạt được nhiều thành tựu trong các nhiệm vụ suy luận và giải quyết bài toán. Tuy nhiên, đối với các bài toán Toán học ở mức Olympic, việc chỉ sử dụng suy luận bằng ngôn ngữ tự nhiên thường chưa đủ để đảm bảo tính chính xác, đặc biệt đối với các phép tính phức tạp hoặc các bài toán yêu cầu lập luận nhiều bước.

Trong Assignment 3, nhóm xây dựng một hệ thống sử dụng mô hình ngôn ngữ lớn kết hợp với các kỹ thuật hỗ trợ nhằm giải các bài toán trong bộ dữ liệu **AI Mathematical Olympiad (AIMO) Progress Prize 3** trên nền tảng Kaggle. Hệ thống áp dụng phương pháp *Program-of-Thought (PoT)*, trong đó mô hình không chỉ sinh lời giải mà còn sinh mã Python để thực hiện các phép tính, từ đó giảm sai số trong quá trình suy luận.

Đầu vào của hệ thống là các bài toán được biểu diễn dưới dạng văn bản (text-only) có sử dụng các công thức bằng LaTeX. Đầu ra là đáp án cuối cùng của bài toán, đồng thời hệ thống có thể lưu lại chuỗi suy luận hoặc đoạn mã Python được sinh ra nhằm phục vụ việc phân tích và đánh giá kết quả.

1.2 Phạm vi dự án

Trong phạm vi của bài tập lớn, nhóm xây dựng một pipeline hoàn chỉnh phục vụ quá trình giải toán bằng mô hình ngôn ngữ lớn. Hệ thống sử dụng mô hình **Qwen2.5-Math-7B-Instruct** kết hợp với kỹ thuật *Self-Consistency* (Majority Voting) nhằm tăng độ ổn định của kết quả dự đoán.

Pipeline được triển khai và kiểm thử trên hai môi trường:

- Môi trường phát triển cục bộ (Local) sử dụng API Groq để kiểm tra luồng xử lý và gỡ lỗi nhanh.
- Môi trường Kaggle Notebook sử dụng GPU T4 để chạy mô hình Qwen2.5-Math-7B thông qua thư viện **Transformers**.

Do giới hạn về thời gian thực hiện cũng như tài nguyên tính toán của bài tập lớn, nhóm không thực hiện huấn luyện lại (fine-tuning) mô hình, cũng không triển khai các phương pháp nâng cao như Formal Verification hoặc Lean4. Mục tiêu chính của dự án là xây dựng được một hệ thống hoạt động ổn định, có khả năng xử lý toàn bộ pipeline từ sinh lời giải, thực thi mã nguồn, tổng hợp kết quả đến đánh giá hiệu năng và phân tích lỗi.

1.3 Phân chia công việc

Để đảm bảo tiến độ và chất lượng của dự án, các thành viên được phân công theo từng module chức năng của hệ thống như Bảng 1.

Bảng 1: Phân công công việc của các thành viên

Thành viên	Công việc thực hiện
Nguyễn Thị Nhật Minh	• Thiết kế Prompt và Few-shot Examples. • Xây dựng Engine kết nối Groq API và mô hình trên Kaggle. • Phát triển module Majority Voting. • Tích hợp hệ thống bằng Git và triển khai trên Kaggle. • Thực hiện đánh giá thực nghiệm, phân tích kết quả và viết báo cáo.
Phạm Thành Trung	• Xây dựng Parser và Python Executor. • Phát triển API Wrapper và Mock Engine. • Xây dựng module đánh giá kết quả. • Chuẩn bị môi trường triển khai Offline trên Kaggle. • Xây dựng Unit Test và các công cụ tự động hóa.

2 CƠ SỞ LÝ THUYẾT VÀ PHƯƠNG PHÁP

2.1 Mô hình ngôn ngữ lớn (Large Language Model)

Trong các bài toán Olympic, LLM không chỉ cần hiểu nội dung đề bài mà còn phải thực hiện quá trình lập luận nhiều bước để tìm ra đáp án cuối cùng. Tuy nhiên, LLM thường gặp khó khăn về tính chính xác của các phép tính phức tạp và không thể kiểm tra logic từng bước khi giải toán. Các phép tính số học hoặc các bài toán yêu cầu suy luận phức tạp thường là điểm hạn chế của các mô hình ngôn ngữ nếu chỉ sử dụng phương pháp sinh văn bản thông thường.

Nhóm sử dụng mô hình **Qwen2.5-Math-7B-Instruct**, một mô hình được huấn

luyện chuyên biệt cho các bài toán toán học. Mô hình hỗ trợ sinh lời giải theo ngôn ngữ tự nhiên cũng như sinh mã Python để hỗ trợ quá trình tính toán.

2.2 Program-of-Thought (PoT)

Program-of-Thought (PoT) là phương pháp yêu cầu mô hình sinh ra đoạn mã chương trình thay vì trực tiếp sinh đáp án cuối cùng. Sau khi được tạo, đoạn mã sẽ được thực thi trong môi trường Python để tính toán kết quả.

So với phương pháp Chain-of-Thought truyền thống, PoT có ưu điểm là các phép tính được thực hiện bởi trình thông dịch Python thay vì dựa hoàn toàn vào khả năng tính toán của mô hình ngôn ngữ. Điều này giúp giảm đáng kể các lỗi số học và tăng tính chính xác đối với các bài toán yêu cầu tính toán nhiều bước.

Trong hệ thống của nhóm, mô hình Qwen2.5-Math sinh ra đoạn mã Python, sau đó đoạn mã được kiểm tra, thực thi và trích xuất kết quả để làm đáp án dự đoán.

2.3 Self-Consistency và Majority Voting

Một trong những hạn chế của LLM là kết quả sinh ra có thể thay đổi giữa các lần suy luận do quá trình lấy mẫu ngẫu nhiên (sampling). Vì vậy, nhóm sử dụng kỹ thuật *Self-Consistency*, trong đó mô hình được yêu cầu sinh nhiều lời giải độc lập cho cùng một bài toán.

Sau khi thực thi tất cả các lời giải, hệ thống thống kê các đáp án hợp lệ và lựa chọn đáp án xuất hiện nhiều nhất thông qua thuật toán *Majority Voting*. Cách tiếp cận này giúp giảm ảnh hưởng của các lời giải bất thường và tăng tính ổn định của hệ thống.

2.4 Kiến trúc mô hình Qwen2.5-Math

Qwen2.5-Math-7B-Instruct là mô hình ngôn ngữ mã nguồn mở được Alibaba phát triển và tối ưu cho các bài toán toán học. So với các mô hình ngôn ngữ thông thường, Qwen2.5-Math được huấn luyện trên lượng lớn dữ liệu toán học và mã nguồn, nhờ đó cải thiện khả năng lập luận cũng như sinh chương trình.

Nhóm lựa chọn phiên bản 7B vì các lý do sau:

- Có thể chạy trên GPU T4 miễn phí của Kaggle.
- Hỗ trợ tốt cho cả suy luận bằng ngôn ngữ tự nhiên và sinh mã Python.
- Có sẵn trên HuggingFace và dễ tích hợp thông qua thư viện **Transformers**.

Việc sử dụng mô hình mã nguồn mở cũng giúp quá trình triển khai và đánh giá được thực hiện hoàn toàn trong môi trường của nhóm mà không phụ thuộc vào các dịch vụ thương mại.

3 THIẾT KẾ HỆ THỐNG

3.1 Kiến trúc tổng thể

Hệ thống được thiết kế theo dạng pipeline, trong đó mỗi thành phần đảm nhận một chức năng riêng biệt và dữ liệu được truyền tuần tự giữa các module. Kiến trúc này giúp các thành phần có tính độc lập cao, thuận tiện cho việc kiểm thử, mở rộng và thay thế từng module khi cần thiết.

Quy trình xử lý của hệ thống như sau:

1. Đọc bài toán từ bộ dữ liệu AIMO.
2. Xây dựng Prompt và gửi đến mô hình Qwen2.5-Math.
3. Mô hình sinh lời giải hoặc mã Python.
4. Parser trích xuất đoạn mã cần thực thi.
5. Python Executor thực hiện chương trình trong môi trường an toàn.
6. Thu được nhiều đáp án từ các lần suy luận khác nhau.
7. Majority Voting tổng hợp các đáp án và lựa chọn kết quả cuối cùng.

3.2 Module Prompt Builder

Prompt Builder đóng vai trò định hình và chuẩn hóa cấu trúc đầu vào (Input Engineering) trước khi gửi tới mô hình ngôn ngữ lớn. Đối với mỗi bài toán trong bộ dữ liệu AIMO, mô-đun này thực hiện kỹ thuật đóng gói ngữ cảnh động kết hợp giữa ba thành phần:

- **System Prompt:** Thiết lập vai trò cố định cho LLM có khả năng giải các bài toán Olympic khó và có thể lập trình Python tốt.
- **Few-shot Examples:** Cung cấp các cặp bài toán - mã nguồn mẫu chính xác để nâng cao khả năng học trong ngữ cảnh (In-context Learning) của mô hình.
- **Format Constraints:** Chỉ thị nghiêm ngặt yêu cầu mô hình phải bao bọc toàn bộ mã nguồn giải toán trong khối định dạng markdown ““python ... “” và không sinh các ký tự dư thừa.

Việc chuẩn hóa này giúp kiểm soát không gian sinh văn bản của LLM, giảm thiểu tối đa hiện tượng phản hồi sai cấu trúc câu cú pháp và tăng tỷ lệ bóc tách thành công ở các bước sau.

3.3 Module Engine

Engine là lớp trừu tượng hóa (Abstraction Layer) chịu trách nhiệm quản lý kết nối và điều phối truy vấn đến mô hình ngôn ngữ lớn, giúp cô lập logic nghiệp vụ của hệ thống với hạ tầng tính toán bên dưới.

- **Giai đoạn phát triển (Development):** Hệ thống tích hợp dịch vụ Groq Cloud API nhằm sử dụng tốc độ suy luận nhanh của chip LPU, phục vụ việc kiểm thử nhanh cấu trúc Prompt, logic của bộ Parser và Executor.
- **Giai đoạn hoàn thiện (Deployment):** Engine được chuyển cấu hình sang chế độ vận hành ngoại tuyến (Offline) trên Kaggle Notebook. Mô hình **Qwen2.5-Math-7B-Instruct** được nạp trực tiếp vào VRAM của GPU T4 thông qua class `AutoModelForCausalLM` của thư viện `Transformers`.

Sự độc lập của module Engine cho phép nhóm dễ dàng chuyển đổi qua lại giữa việc gọi API thương mại và chạy mô hình local chỉ bằng cách thay đổi tham số cấu hình hệ thống.

3.4 Module Parser

Đầu ra từ mô hình Qwen2.5-Math thường là một chuỗi văn bản bao gồm cả lời giải thích tự nhiên và mã nguồn. Mô-đun Parser được xây dựng dựa trên các biểu thức chính quy (Regular Expression - Regex) để quét và trích xuất phân đoạn dữ liệu cần thiết.

Nhiệm vụ cốt lõi của Parser là định vị cặp thẻ `“python` và `”` để bóc tách khối mã nguồn Python nằm bên trong. Một số kịch bản xử lý ngoại lệ của Parser bao gồm:

- **Trường hợp lý tưởng:** Trích xuất thành công một khối mã chuẩn và chuyển tiếp trực tiếp sang module Executor.
- **Trường hợp sinh nhiều khối code:** Parser được cấu hình để tự động nối ghép (concatenate) các khối hoặc chỉ lựa chọn khối mã cuối cùng - nơi thường chứa hàm xuất kết quả.
- **Trường hợp không có mã nguồn:** Nếu LLM chỉ trả về văn bản thuần túy, Parser sẽ ném ra ngoại lệ `ParsingError`, đánh dấu lượt sinh đó không hợp lệ để hệ thống xử lý ở các bước sau.

3.5 Module Python Executor

Sau khi nhận được chuỗi mã nguồn sạch từ Parser, hệ thống chuyển giao cho Python Executor để tiến hành thực thi động (Dynamic Execution). Nhằm tránh các rủi ro bảo mật tiềm ẩn cũng như hiện tượng treo hệ thống do mã nguồn AI tự sinh, Executor được trang bị các cơ chế kiểm soát chặt chẽ:

- **Giới hạn thời gian (Timeout):** Sử dụng luồng quản lý ngoại vi (`subprocess`) để giới hạn thời gian chạy của mỗi đoạn mã tối đa là 5 giây. Mọi chương trình rơi vào vòng lặp vô hạn sẽ bị ngắt cưỡng bức và ghi nhận lỗi `TimeoutError`.
- **Bắt lỗi runtime (Exception Handling):** Module bọc tiến trình trong khối `try-except` để bắt toàn bộ các lỗi nội tại của Python như `SyntaxError` (sai cú pháp), `ZeroDivisionError` (chia cho 0), hoặc `NameError` (biến chưa khai báo).
- **Đánh chặn đầu ra:** Executor đánh chặn luồng xuất chuẩn (`stdout`), trích xuất giá trị số được in ra cuối cùng và ép kiểu về dạng số nguyên theo đúng quy chế của cuộc thi AIMO.

3.6 Module Majority Voting

Để khắc phục nhược điểm mất ổn định định dạng và tính ngẫu nhiên do quá trình lấy mẫu hình học (*sampling*) của LLM, hệ thống áp dụng kỹ thuật *Self-Consistency* thông qua cơ chế bỏ phiếu Majority Voting.

Đối với mỗi bài toán, hệ thống yêu cầu mô hình sinh độc lập n lời giải (với tham số $n = 3$). Sau khi đi qua bộ lọc Parser và thực thi bởi Executor, hệ thống thu được một tập hợp các đáp án thô:

$$A = \{a_1, a_2, \dots, a_n\}$$

Đáp án cuối cùng được đưa ra dựa trên kết quả của hàm tìm giá trị *Mode* theo công thức toán học:

$$\text{Prediction} = \arg \max_x \text{Count}(x)$$

Trong đó, $\text{Count}(x)$ là hàm đếm tần suất xuất hiện của giá trị đáp án x trong tập hợp A . Kỹ thuật này giúp hệ thống tự động loại bỏ các câu trả lời đột biến bị sai do mô hình hallucination ở một lượt sinh bất kỳ.

3.7 Cơ chế dự phòng và Xử lý ngoại lệ tổng thể (Fallback Mechanism)

Để đảm bảo hệ thống vận hành ổn định và chắc chắn không gặp lỗi dừng chương trình (*Submission Error*) khi chấm điểm tự động trên Kaggle, pipeline được trang bị cơ chế dự phòng hai lớp:

- **Lớp dự phòng 1 (Text Fallback):** Nếu toàn bộ n lượt sinh mã của bài toán đều thất bại ở bước Parser hoặc Executor (tập hợp $A = \emptyset$), hệ thống sẽ tự động kích hoạt một Prompt phụ, yêu cầu mô hình giải bài toán bằng phương pháp chuỗi tư duy truyền thống (Zero-shot CoT) và trích xuất đáp án từ thẻ `\boxed{}`.
- **Lớp dự phòng 2 (Hard Fallback):** Trong trường hợp xấu nhất khi cả phương pháp dự phòng bằng văn bản vẫn không thể đưa ra kết quả, pipeline sẽ tự động trả về giá trị mặc định là 0 nhằm bảo toàn cấu trúc tệp xuất dữ liệu `submission.csv`.

4 THỰC NGHIỆM VÀ KẾT QUẢ

4.1 Môi trường thực nghiệm

Hệ thống được phát triển và kiểm thử cục bộ trên máy tính cá nhân thông qua Groq Cloud API, sau đó triển khai trên Kaggle Notebook để đánh giá chính thức với mô hình Qwen2.5-Math-7B-Instruct chạy ngoại tuyến.

Bảng 2 trình bày cấu hình môi trường thực nghiệm.

Bảng 2: Môi trường thực nghiệm

Thành phần	Môi trường cục bộ	Kaggle Notebook
Hệ điều hành	Windows 11	Linux (Ubuntu)
Python	3.13	3.10
Mô hình LLM	Groq Cloud API	Qwen2.5-Math-7B-Instruct
Inference	LPU (Groq)	Transformers / vLLM
GPU	Không yêu cầu	Kaggle T4
Phương pháp	Program-of-Thought	Program-of-Thought + Majority Voting

Trong giai đoạn phát triển cục bộ, nhóm sử dụng Groq Cloud API để kiểm tra nhanh các thành phần của hệ thống bao gồm Prompt, Parser và Executor, tận dụng tốc độ suy luận cao của kiến trúc chip LPU. Sau khi hoàn thiện pipeline, toàn bộ quá trình đánh giá chính thức được thực hiện trên Kaggle nhằm đảm bảo thống nhất môi trường thực thi với mô hình toán học chuyên biệt Qwen2.5-Math-7B-Instruct, đồng thời đáp ứng yêu cầu chạy ngoại tuyến (offline) của cuộc thi.

4.2 Bộ dữ liệu

Nhóm sử dụng bộ dữ liệu AI Mathematical Olympiad (AIMO Progress Prize 3) do Kaggle cung cấp, bao gồm các bài toán Olympic Toán học ở mức độ AIME (American Invitational Mathematics Examination).

Bộ dữ liệu gồm hai thành phần chính:

- **reference.csv**: gồm các bài toán có đáp án dùng để đánh giá cục bộ.
- **test.csv**: dùng để kiểm thử pipeline trước khi nộp lên Kaggle.

Mỗi bài toán được biểu diễn dưới dạng văn bản có chứa các biểu thức toán học viết bằng cú pháp **Latex**. Đầu ra mong muốn là một số nguyên trong khoảng $[0, 999]$ biểu diễn đáp án cuối cùng của bài toán (lấy phần dư khi chia cho 1000).

4.3 Thiết lập thực nghiệm

Đối với mỗi bài toán, hệ thống thực hiện các bước sau:

1. Đọc đề bài từ bộ dữ liệu.
2. Phân loại bài toán qua bộ định tuyến **MathRouter** (hình học hoặc đại số/khác).
3. Sinh Prompt phù hợp với loại bài toán.
4. Gửi Prompt tới mô hình ngôn ngữ.
5. Mô hình sinh mã Python (PoT) hoặc lời giải văn bản (CoT đối với bài hình học).
6. **ResponseParser** trích xuất mã nguồn Python.
7. **PythonExecutor** thực thi mã trong môi trường sandbox an toàn.
8. Lặp lại nhiều lần để thu thập các lời giải độc lập.
9. **Majority Voting** lựa chọn đáp án xuất hiện nhiều nhất.

Các tham số chính của hệ thống được trình bày trong Bảng 3.

Bảng 3: Các tham số thực nghiệm

Tham số	Giá trị
Temperature	0.7
Top-p	0.95
Max Tokens	2048
Số lượt Majority Voting	3
Timeout thực thi (giây)	10
Mô hình (Kaggle)	Qwen2.5-Math-7B-Instruct

4.4 Kết quả thực nghiệm

Trong giai đoạn kiểm thử cục bộ, nhóm đã xây dựng và xác minh thành công toàn bộ pipeline xử lý, bao gồm các thành phần: sinh Prompt, gọi mô hình ngôn ngữ (LLM), trích xuất mã nguồn (Parser), thực thi mã an toàn trong môi trường sandbox (Executor), và tổng hợp đáp án bằng Majority Voting. Hệ thống được thiết kế linh hoạt với khả năng hỗ trợ nhiều nhà cung cấp mô hình khác nhau (Groq, Google Gemini, Anthropic Claude và OpenAI) thông qua một lớp giao tiếp trừu tượng (**LLMEngineAPI**), cho phép dễ dàng chuyển đổi giữa các backend mà không cần thay đổi logic chính.

Kết quả đánh giá trên một mẫu gồm 10 bài toán đầu tiên của tập dữ liệu được trình bày trong Bảng 4.

Bảng 4: Kết quả thực nghiệm (mẫu 10 bài toán)

STT	ID bài toán	Dạng bài	Trạng thái pipeline
1	bcd0177b	Hình học	Thực thi thành công
2	ad3905ac	Đại số	Thực thi thành công
3	f472765e	Đại số	Thực thi thành công
4	73880ce1	Hình học	Thực thi thành công
5	2541e516	Tổ hợp	Thực thi thành công
6	12ae0a4c	Số học	Thực thi thành công
7	ed1d6794	Hình học	Thực thi thành công
8	b1d2585d	Tổ hợp	Thực thi thành công
9	22d623bc	Hình học	Thực thi thành công
10	b8ea09b7	Xác suất	Thực thi thành công

Kết quả cho thấy pipeline vận hành ổn định và hoàn thành đầy đủ các bước xử lý trên toàn bộ 10 bài toán mẫu, không phát sinh lỗi dừng chương trình. Hệ thống có khả năng phân loại bài toán, sinh mã, thực thi và tổng hợp đáp án một cách tự động.

Cần lưu ý rằng độ chính xác cuối cùng phụ thuộc chủ yếu vào năng lực suy luận của mô hình ngôn ngữ được sử dụng. Trong giai đoạn phát triển cục bộ, do giới hạn về số lượng yêu cầu (rate limit) và tín dụng của các API miễn phí, việc đánh giá quy mô lớn về độ chính xác được chuyển sang môi trường Kaggle với mô hình Qwen2.5-Math-7B-Instruct chạy ngoại tuyến, đảm bảo không phụ thuộc vào giới hạn của dịch vụ bên ngoài.

4.5 Đánh giá ảnh hưởng của Majority Voting

Để đánh giá hiệu quả của kỹ thuật Majority Voting, nhóm tiến hành so sánh về mặt lý thuyết và quan sát thực nghiệm với phương pháp chỉ sinh một lời giải duy nhất (Single Inference).

Quan sát trong quá trình thực nghiệm cho thấy Majority Voting giúp giảm ảnh hưởng của các lần sinh ngẫu nhiên và cải thiện tính ổn định của hệ thống. Trong một số trường hợp, mặc dù một vài lời giải riêng lẻ cho kết quả sai (do hiện tượng hallucination ở một lượt sinh bất kỳ), đáp án đúng vẫn được lựa chọn nhờ xuất hiện với tần suất cao nhất trong tập kết quả.

Tuy nhiên, phương pháp này đi kèm với chi phí thời gian cao hơn do cần thực hiện nhiều lần inference. Trong môi trường Kaggle với giới hạn thời gian nộp bài, nhóm cân bằng giữa số lượt voting ($n = 3$) và thời gian thực thi để đảm bảo hoàn thành trong thời gian cho phép.

4.6 Thảo luận

Qua các thực nghiệm, nhóm nhận thấy hệ thống hoạt động ổn định đối với các bài toán yêu cầu tính toán đại số hoặc số học. Tuy nhiên, đối với các bài toán hình học hoặc tổ hợp phức tạp, mô hình vẫn gặp khó khăn trong việc xây dựng lập luận chính xác.

Ngoài ra, chất lượng của kết quả phụ thuộc đáng kể vào Prompt cũng như khả năng sinh mã Python của mô hình. Nếu mô hình sinh mã không hợp lệ hoặc hiểu sai yêu cầu của đề bài thì toàn bộ pipeline sẽ không thể đưa ra đáp án chính xác, mặc dù các module Parser và Executor vẫn hoạt động đúng chức năng.

5 PHÂN TÍCH LỖI

Mặc dù hệ thống đã xây dựng thành công pipeline hoàn chỉnh cho bài toán giải toán Olympic bằng mô hình ngôn ngữ lớn, kết quả thực nghiệm cho thấy vẫn tồn tại nhiều trường hợp dự đoán chưa chính xác. Trong quá trình đánh giá, nhóm tiến hành phân loại các lỗi phát sinh theo từng thành phần của hệ thống nhằm xác định nguyên nhân và đề xuất hướng cải thiện.

5.1 Phân loại lỗi

Các lỗi được chia thành bốn nhóm chính như Bảng 5.

Bảng 5: Phân loại các nhóm lỗi

Loại lỗi	Mô tả
Formatting Error	Mô hình không sinh đúng định dạng mong muốn hoặc không sinh được mã Python.
Logical Error	Mô hình hiểu sai yêu cầu của đề bài hoặc xây dựng lời giải chưa đầy đủ.
Algorithmic Error	Thuật toán sinh ra không phù hợp, dẫn đến thời gian thực thi lớn hoặc kết quả sai.
System Error	Các lỗi phát sinh trong quá trình Parser hoặc Executor xử lý đầu ra của mô hình.

5.2 Formatting Error

Đây là nhóm lỗi xuất hiện nhiều nhất trong quá trình thực nghiệm.

Trong nhiều trường hợp, mô hình sinh lời giải dưới dạng văn bản tự nhiên thay vì đoạn mã Python như yêu cầu của Prompt. Khi đó Parser không thể trích xuất được mã nguồn để thực thi.

Ví dụ:

```
"The answer is \boxed{42}."
```

Trong trường hợp này, hệ thống không thu được đoạn mã Python hợp lệ nên lời giải bị loại khỏi quá trình Majority Voting.

Nguyên nhân chủ yếu là do mô hình chưa luôn tuân thủ định dạng đầu ra mặc dù Prompt đã quy định rõ.

5.3 Logical Error

Logical Error xảy ra khi mô hình sinh ra chương trình hợp lệ về mặt cú pháp nhưng thuật toán giải bài toán không chính xác.

Ví dụ, mô hình có thể bỏ sót điều kiện của đề bài, hiểu sai giả thiết hoặc sử dụng công thức không phù hợp.

Trong các trường hợp này:

- Parser hoạt động đúng.
- Executor thực thi thành công.
- Chương trình không phát sinh ngoại lệ.
- Tuy nhiên đáp án cuối cùng vẫn sai.

Đây là nhóm lỗi khó khắc phục nhất vì xuất phát từ khả năng suy luận của chính mô hình ngôn ngữ.

5.4 Algorithmic Error

Một số bài toán Olympic yêu cầu thuật toán tối ưu hoặc có không gian tìm kiếm rất lớn.

Trong khi đó, mô hình đôi khi sinh ra các lời giải theo hướng vét cạn (brute-force).

Điều này dẫn tới:

- thời gian thực thi lớn;
- vượt quá giới hạn thời gian;
- tiêu tốn nhiều tài nguyên tính toán;
- hoặc không thể hoàn thành chương trình.

Đối với các trường hợp này, hệ thống ghi nhận lỗi và loại bỏ lời giải khỏi quá trình bỏ phiếu.

5.5 System Error

Bên cạnh các lỗi do mô hình sinh ra, nhóm cũng ghi nhận một số lỗi thuộc về hệ thống. Ví dụ:

- Parser nhận diện nhầm đoạn văn bản là mã Python.
- Mã nguồn bị cắt không đầy đủ.
- Chương trình phát sinh `SyntaxError` khi thực thi.
- Timeout do chương trình chạy quá lâu.

Các lỗi này không phản ánh khả năng suy luận của mô hình mà xuất phát từ quá trình xử lý của pipeline.

Sau quá trình kiểm thử, nhóm đã cải thiện Parser để chỉ trích xuất các khối mã nằm trong cặp ký hiệu `“python ... ”`, qua đó giảm đáng kể số lượng lỗi nhận diện sai.

5.6 Đánh giá tổng quát

Qua quá trình thực nghiệm, nhóm nhận thấy phần lớn lỗi phát sinh không nằm ở Executor mà chủ yếu xuất hiện trong giai đoạn sinh lời giải của mô hình.

Pipeline được xây dựng hoạt động ổn định và có khả năng xử lý đầy đủ các bước từ sinh lời giải, thực thi chương trình đến tổng hợp kết quả. Tuy nhiên, chất lượng của kết quả cuối cùng vẫn phụ thuộc chủ yếu vào khả năng lập luận của mô hình Qwen2.5-Math.

Kết quả phân tích lỗi cho thấy việc cải thiện Prompt, tăng chất lượng của mã nguồn sinh ra và sử dụng các mô hình có khả năng suy luận mạnh hơn sẽ là những hướng nâng cao hiệu quả của hệ thống trong tương lai.

6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Trong bài toán này, nhóm đã xây dựng một hệ thống giải toán Olympic dựa trên mô hình ngôn ngữ lớn Qwen2.5-Math-7B-Instruct kết hợp với phương pháp Program-of-Thought và kỹ thuật Majority Voting.

Hệ thống được thiết kế theo dạng pipeline gồm các thành phần chính như Prompt Builder, Engine, Parser, Python Executor và Majority Voting. Mỗi module đảm nhận một chức năng riêng biệt, giúp quá trình phát triển, kiểm thử và mở rộng hệ thống trở nên thuận tiện hơn.

Thông qua các thực nghiệm trên bộ dữ liệu AI Mathematical Olympiad (AIMO Progress Prize 3), nhóm đã triển khai thành công toàn bộ quy trình từ tiếp nhận bài

toán, sinh lời giải, thực thi mã Python đến tổng hợp đáp án cuối cùng. Kết quả thực nghiệm cho thấy việc kết hợp Program-of-Thought với Majority Voting giúp hệ thống hoạt động ổn định hơn so với việc chỉ sinh một lời giải duy nhất.

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế. Độ chính xác của mô hình chưa cao đối với các bài toán yêu cầu lập luận phức tạp hoặc có không gian tìm kiếm lớn. Ngoài ra, chất lượng kết quả vẫn phụ thuộc nhiều vào Prompt cũng như khả năng sinh mã Python của mô hình ngôn ngữ.

Mặc dù chưa đạt được hiệu năng cao trên các bài toán khó, dự án đã hoàn thành đầy đủ các yêu cầu của bài tập lớn, xây dựng được một hệ thống hoạt động hoàn chỉnh và thực hiện được quá trình đánh giá cũng như phân tích các loại lỗi phát sinh trong quá trình suy luận.

6.2 Hướng phát triển

Trong tương lai, hệ thống có thể được cải thiện theo một số hướng sau:

- Tối ưu Prompt nhằm tăng khả năng mô hình sinh đúng định dạng và giảm số lượng lỗi trong quá trình suy luận.
- Thử nghiệm các mô hình ngôn ngữ có năng lực suy luận mạnh hơn như DeepSeek-Math hoặc các phiên bản Qwen mới hơn để cải thiện độ chính xác.
- Bổ sung các công cụ hỗ trợ suy luận ký hiệu (Symbolic Solver) nhằm giải quyết tốt hơn các bài toán đại số và tổ hợp.
- Cải tiến Parser và Python Executor để xử lý được nhiều trường hợp đầu ra phức tạp hơn, đồng thời tăng tính ổn định của hệ thống.
- Thực hiện đánh giá trên nhiều bộ dữ liệu toán học khác nhằm kiểm chứng khả năng tổng quát hóa của hệ thống.